Simulating the uNPU from scratch

From one xrun command to gate-level simulation of the synthesised netlist, with every command used in the project

EE5239 · Project 2 · Frankenstein 180 nm IP-Validation SoC
Matches `sim/`, `gls/`, `scripts/run_xrun.sh` and the Xcelium history files; results from `docs/session-handoff.md` §18.7–18.8
November 2026

What is inside

1. What a logic simulator actually does
2. Your first run, step by step
3. Inside `xrun`: compile, elaborate, simulate
4. Running every testbench
5. Time, events, and the `#1` in every testbench
6. Looking at waveforms
7. Two simulators: Verilator and Xcelium
8. Gate-level simulation of the netlist
9. Post-layout simulation with an SDF (next step)
10. Troubleshooting
11. Command reference card

1. What a logic simulator actually does

A simulator does not build hardware. It keeps a list of **events** ("signal `x` changes to 1 at time 120 ns") and processes them in time order. When a signal changes, every piece of code sensitive to it (an `always` block, an `assign`) runs and may schedule more events. Simulated time only advances when nothing is left to do at the current time.

Three properties of this model matter for the uNPU:

- **Four values, not two.** Every bit is 0, 1, X (unknown) or Z (undriven). After power-up a flip-flop is X until reset. An X that reaches an output is a bug signal, and the testbenches compare with `!==` so an X can never slip through as a pass.
- **Time has units.** `` `timescale 1ns/1ps `` means `#5` is 5 ns, resolved to 1 ps. The testbench clock toggles every 5 ns (a 10 ns period); the simulated speed is irrelevant to RTL behaviour.
- **Zero-delay RTL.** In RTL, logic has no delay: a flip-flop's new value is visible the instant after the edge. Real gates have delays, which only gate-level simulation (chapter 8) adds.

The uNPU project uses two simulators: **Cadence Xcelium 22.09** (`xrun`) on the institute server, and **Verilator 5.053** on a laptop.

2. Your first run, step by step

Everything runs **from the repository root**: the testbenches open `model/vectors/...` by relative path.

```
cd /storage/EE5239G1/<your_work_dir>/unpu_flow     # your clone on the server
git pull --ff-only
source ~/cad_cshrc                                  # loads Xcelium (PATH + licence)
which xrun                                          # .../XCELIUM2209/.../xrun
gcc -std=c99 -Wall -Wextra -o model/golden model/golden.c
./model/golden                                      # ends: ALL CASES PASSED AND WRITTEN
```

Now simulate the processing-element testbench. This is the exact command recorded in `sim/pe/history` on 15 Oct 2026:

```
xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_pe_tb \
     -xmlibdirname sim/pe -l logs/sim_pe.log -f scripts/filelist.f tb/unpu_pe_tb.sv
```

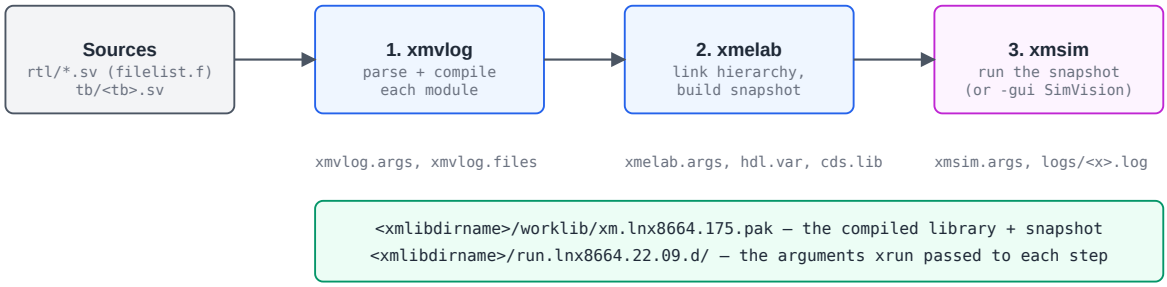
Then read the verdict:

```
grep -E "ALL .*PASSED" logs/sim_pe.log     # ALL TASK 013 + TASK 019 PE CHECKS PASSED
...
grep -cE "\*E,|\*F," logs/sim_pe.log       # 0  (Xcelium errors / fatals)
grep -c FAIL logs/sim_pe.log               # 0  (failed checks)
```

All three conditions together mean the run passed. Don’t judge by xrun’s exit code: the testbench ends with `$finish`, which exits 0 even after failed checks.

3. Inside `xrun`: compile, elaborate, simulate

What one xrun command does: compile → elaborate → simulate



-xmlibdirname picks the folder: sim/compile, sim/pe, gls/top, xrun_out/<tb>. Everything in it is regenerated on the next run.

-c stops after step 2 (compile/elaborate check only). -gui opens SimVision at step 3. -l writes the log that decides PASS/FAIL.

xrun also appends every invocation to <xmlibdirname>/history — the most reliable record of what was actually run.

One xrun command runs three tools

`xrun` is a driver for three tools:

1. **`xmvlog`** reads and compiles every source file into a library (`worklib`). Syntax errors stop here.
2. **`xmelab`** starts at `-top`, connects every instance below it, checks port widths, and writes a **snapshot**: the whole design, flattened and ready to run.
3. **`xmsim`** loads the snapshot and simulates it, in batch or, with `-gui`, inside SimVision.

`-xmlibdirname` chooses the folder all of this goes in, which is why the repository has `sim/compile/`, `sim/pe/` and `gls/top/`:

Where simulation output lands (all regenerable, none of it is source)

`sim/compile/`	RTL compile-only check of unpu_top `xrun ... -c -top unpu_top`
`sim/pe/`	PE testbench, batch and GUI runs `xrun ... -top unpu_pe_tb`
`gls/top/`	gate-level sim of the DC netlist `xrun ... -notimingchecks -top unpu_top_tb`
`xrun_out/<tb>/`	scripts/run_xrun.sh regression `one lib dir + .log + .console per TB`
`logs/`	xrun -l logs and copies of history `compile.log, sim_pe.log, gls_top.log`
`waves.shm/`	SimVision waveform database `written by the -gui / probe session`
`.simvision/`	SimVision bookmarks `created by opening the GUI`

After scripts/cleanup_repo.ps1 these build folders are git-ignored; only the small 'history' files and this README stay tracked.

What lives where

Inside each one, `run.lnx8664.22.09.d/` keeps the exact arguments passed to each step (`xmvlog.args`, `xmelab.args`, `xmsim.args`), the absolute paths of the compiled files (`xmvlog.files`), and the library map (`cds.lib`, `hdl.var`). `worklib/xm.lnx8664.175.pak` is the compiled design. The small **`history`** file records every command ever run against that folder, with a timestamp:

```
s1::(15Oct2026:20:01:01):( xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_pe_tb
     -xmlibdirname sim/pe -l logs/sim_pe.log -f scripts/filelist.f tb/unpu_pe_tb.sv )
s2::(15Oct2026:20:03:04):( ... the same ... -gui )
```

Everything except `history` is regenerated by the next run and can be deleted.

The flags, one by one:

Flag	Why it is there
`-sv`	Sources are SystemVerilog (`logic`, `always_ff`, packed arrays)
`-64bit`	64-bit simulator
`-access +r`	Keep every signal readable. Without it, Xcelium may optimise signals away; the testbenches' hierarchical probes and all waveforms need it
`-timescale 1ns/1ps`	The RTL files have no `` `timescale ``; this gives them the same one as the testbenches
`-top unpu_pe_tb`	Elaborate from this module
`-f scripts/filelist.f`	The 11 RTL files, one per line
`-l logs/sim_pe.log`	The log the verdict is read from
`-c`	Compile and elaborate, but don't run (a quick syntax/connectivity check)

The compile-only check of the whole RTL, from `sim/compile/history`:

```
        xrun -sv -64bit -c -top unpu_top -f scripts/filelist.f -xmlibdirname sim/compile -l
logs/compile.log
```

4. Running every testbench

Every testbench uses the same pattern: change `-top`, the testbench file, the lib dir and the log name.

```
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_grid_tb  -xmlibdirname sim/grid
-l logs/sim_grid.log  -f scripts/filelist.f tb/unpu_grid_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_seq_tb   -xmlibdirname sim/seq
-l logs/sim_seq.log   -f scripts/filelist.f tb/unpu_seq_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_top_tb   -xmlibdirname sim/top
-l logs/sim_top.log   -f scripts/filelist.f tb/unpu_top_tb.sv
```

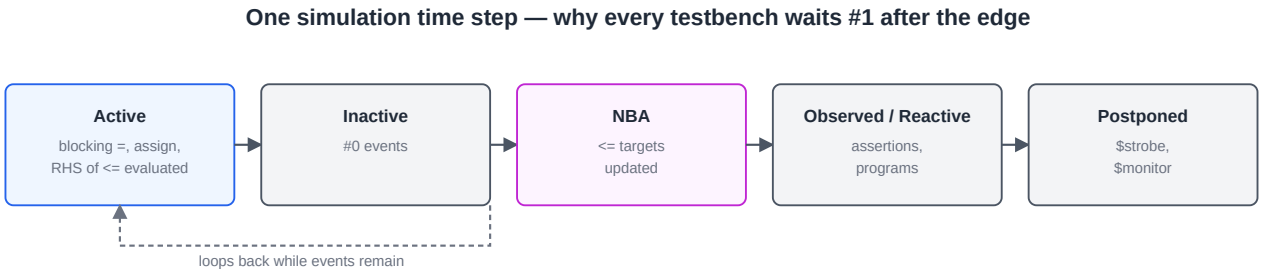
(The full list of twelve is in chapter 11.)

Typing twelve commands and grepping twelve logs is what `scripts/run_xrun.sh` automates:

```
        bash scripts/run_xrun.sh                # pe grid skew stall buf dma csr seq apb top
        bash scripts/run_xrun.sh dma seq top    # a subset
        bash scripts/run_xrun.sh ext1 ext2      # the adapted peer testbenches
        echo $?                                 # 0 all passed · 1 a failure · 2 setup problem
```

For each testbench it runs the command above with `-xmlibdirname xrun_out/<tb>`, then declares FAIL if the log contains `*E,`, `*F,`, `FAIL` or `$fatal`, if it lacks that testbench's own ALL-PASSED line, or if there is no log at all. It also refuses to start if `xrun` is not on the PATH or `model/vectors/` is empty.

5. Time, events, and the `#1` in every testbench



@(posedge clk) wakes the testbench in the Active region of that edge — before the DUT's <= updates have landed.
Checking there would read the OLD register values. The extra #1 moves the check 1 ns later, after the NBA region.

```
task automatic step; @(posedge clk); #1; endtask
```

Inputs driven after the #1 are stable for a full cycle before the next edge — no races between testbench and DUT.

Inside one time step

At a clock edge, every `always_ff` block first **reads** its inputs; only afterwards, in the NBA (non-blocking assignment) region, do the `<=` assignments take effect. That ordering is what makes `q2 <= q1; q1 <= d;` shift by exactly one stage per edge, whatever order the simulator runs the blocks in.

A testbench waiting on `@(posedge clk)` wakes up in the same Active region, **before** the NBA updates. If it checked an output there, it would see the previous cycle's value, and the result could even differ between simulators. So every testbench advances time with:

```
task automatic step;  @(posedge clk);  #1;  endtask
```

The `#1` moves the check 1 ns past the edge, after every register has settled. Inputs driven there are stable for the rest of the cycle, so the design always samples settled values at the next edge. The testbench and the design never race.

6. Looking at waveforms

Interactive. Add `-gui` (this is the second command in `sim/pe/history`). SimVision opens, the design hierarchy is on the left; select signals, *Send To → Waveform*, then press *Run*.

```
xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_pe_tb -xmlibdirname sim/pe \
     -l logs/sim_pe.log -f scripts/filelist.f tb/unpu_pe_tb.sv -gui
```

Batch, then view. `sim/waves.tcl` makes the simulator record everything into an SHM database and quit:

```
database -open waves -shm -default
probe -create -all -depth all -database waves
run
database -close waves
exit
```

```
xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_seq_tb -xmlibdirname sim/seq \
     -l logs/sim_seq.log -f scripts/filelist.f tb/unpu_seq_tb.sv -input sim/waves.tcl
simvision waves.shm &
```

A good first view of one job in `unpu_seq_tb`: `u_seq.state`, `u_seq.cycle`, `u_dma.state`, `dma_valid`, `dma_ready`, `dma_addr`, `u_seq.array_en`, `u_seq.rd_row`, `deskew_c_out`. You should see the state sequence W_FETCH → W_SWAP → A_FETCH → A_SWAP → COMPUTE → WRITE_OUTPUT, with `dma_valid` held high whenever `dma_ready` is low.

For `unpu_top_tb`, which simulates 138 ms, reduce `-depth all` to a few levels, or the database becomes very large.

7. Two simulators: Verilator and Xcelium

The testbenches were developed on **Verilator**: free, fast, runs anywhere. It compiles the design into a C++ program:

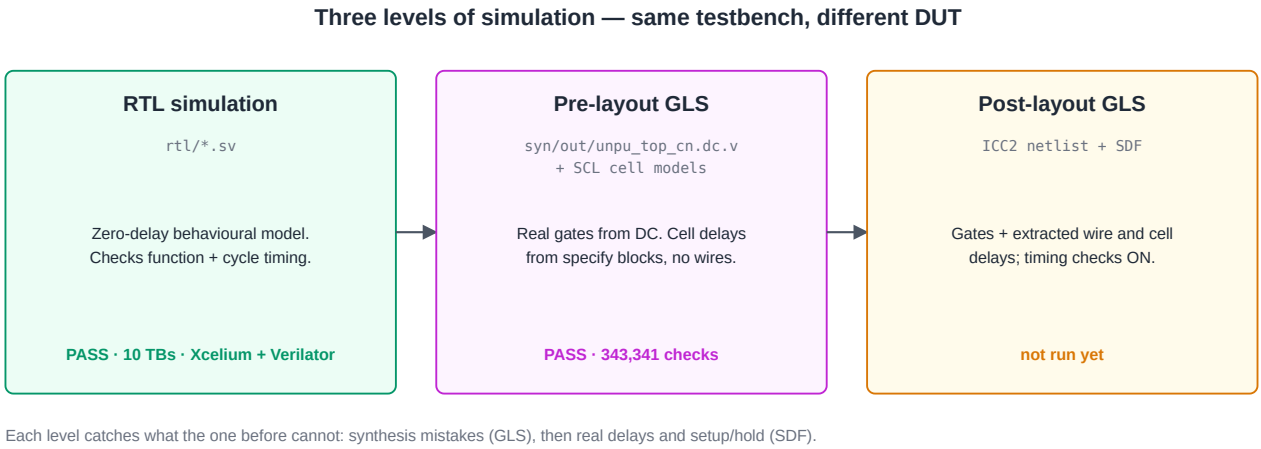
```
        verilator --binary --timing -Wno-fatal --top-module unpu_pe_tb --Mdir obj_dir/pe
rtl/*.sv tb/unpu_pe_tb.sv
        ./obj_dir/pe/Vunpu_pe_tb
        verilator --lint-only -Wall --top-module unpu_top rtl/*.sv    # 4 GENUNNAMED warnings,
all in unpu_grid.sv
```

Verilator is **two-state** (X becomes 0 or 1) and forgiving about some language rules. The first full run on **Xcelium**, four-state and strict IEEE 1800, found three testbench defects that Verilator had hidden:

Xcelium found	Why Verilator missed it
Variables used before their declaration	Verilator does not enforce declaration order
Testbench SRAM model too small: out-of-range writes	Verilator wrapped the index, so writes and reads aliased consistently and the tests passed
`$random(seed)` gave different stall lengths	Each simulator implements `$random` differently

After the fixes, both simulators report identical check counts on every testbench. The rule this project follows: develop on Verilator, **sign off on Xcelium**.

8. Gate-level simulation of the netlist



Same testbench, three different designs under test

Synthesis (Design Compiler) turns the RTL into a **netlist**: about 2 465 flip-flops and thousands of gates from the SCL 180 nm library, connected by wires. Synthesis can go wrong in ways the RTL never shows: a misread construct, a `don't care` resolved differently, a reset that does not reach a flop. **Gate-level simulation (GLS)** catches these by running the same black-box testbench against the netlist.

The exact command, from `gls/top/history`:

```
        xrun -sv -64bit -access +r -timescale 1ns/1ps -notimingchecks -top unpu_top_tb \
             -xmlibdirname gls/top -l logs/gls_top.log \
             syn/out/unpu_top_cn.dc.v \

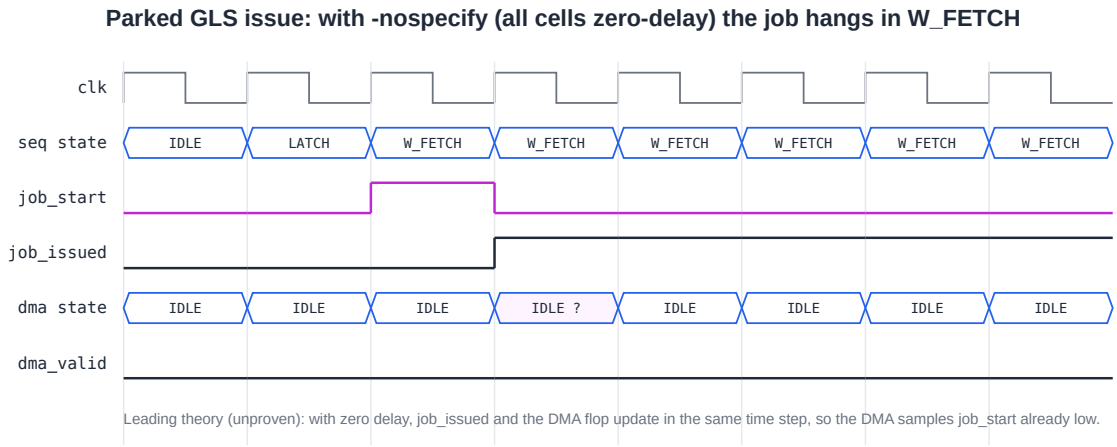
/storage/PDK_iitg/SCLPDK_V3.0_KIT/scl180/stdcell/fs120/4M1IL/verilog/vcs_sim_model/tsl18fs120_scl
.v \
             tb/unpu_top_tb.sv
```

What changed compared with an RTL run:

- **No `filelist.f`.** The RTL is replaced by `syn/out/unpu_top_cn.dc.v`, the cleaned DC netlist: no escaped names, no `assign`, ready for ICC2.
- **Cell models.** Every gate in the netlist (an `ADD`, a `DFF`, a `MUX`) is a module defined in `tsl18fs120_scl.v`. Pass only this combined file; the per-cell files define the same modules again.
- **Only `unpu_top_tb`.** It touches only the APB and DMA ports. Testbenches that probe internal signals by name cannot run on a netlist, because synthesis renames and dissolves the hierarchy.
- **`-notimingchecks`.** The cell models contain `specify` blocks with gate delays and setup/hold checks. The delays stay **on**, making the run more realistic; the checks are turned **off**, because without wire delays (no SDF yet) they would report meaningless violations.

Result. `ALL CHECKS PASSED` with **343 341 checks**, the same number as the RTL run. `$finish` came at 138 350 796 ns, after 1 min 57 s. The run was repeated on the regenerated netlist on 14 Oct with an identical result. The only messages were 24 `CUVWSP` (unused output ports DC left unconnected) and one `UDPRDT` (a redundant row in a cell model's truth table), both benign.

A trap: `-nospecify`. That flag removes every cell delay. With it, the same run **fails**: the sequencer reaches W_FETCH and the DMA never starts.



With every cell at zero delay, the DMA never sees job_start

The leading theory, still unproven, is a zero-delay race. `job_start` is a one-cycle combinational pulse (`state == W_FETCH && !job_issued`). With zero-delay gates, the flop that sets `job_issued` and the DMA's state flop update in the same instant, and the DMA can sample `job_start` already low. With real cell delays the pulse arrives cleanly. Neither the RTL nor the netlist is implicated, since both the RTL run and the run with delays pass. The practical rule: **don't use `-nospecify` for this design.**

A note on VCS. Synopsys VCS refuses to compile `unpu_top_tb.sv` (`ICPD`: the SRAM model `mem` and the back-pressure generator `bp_rng` are written both from an `always_ff` and from other processes). Xcelium accepts this, so GLS uses Xcelium.

9. Post-layout simulation with an SDF (next step)

After place and route, ICC2 knows the length of every wire. It writes a routed netlist and an **SDF** (Standard Delay Format) file listing the real delay of every cell and wire. An SDF-annotated GLS with timing checks **on** is the last simulation before sign-off. It catches setup and hold problems in the context of real stimulus.

This has not been run yet. When ICC2 produces the files, the steps are (file names are placeholders):

```
        # 1. compile the SDF
        xmsdfc pnr/out/unpu_top.sdf -output pnr/out/unpu_top.sdf.X
```

```
# 2. sim/sdf_post.cmd
COMPILED_SDF_FILE = "pnr/out/unpu_top.sdf.X",
SCOPE = unpu_top_tb.dut,
LOG_FILE = "logs/sdf_post.log",
MTM_CONTROL = "MAXIMUM";
```

```
        # 3. simulate with timing checks ON (no -notimingchecks)
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_top_tb \
             -xmlibdirname gls/post -l logs/gls_post.log -sdf_cmd_file sim/sdf_post.cmd \
             pnr/out/unpu_top.pnr.v \

/storage/PDK_iitg/SCLPDK_V3.0_KIT/scl180/stdcell/fs120/4M1IL/verilog/vcs_sim_model/tsl18fs120_scl
.v \
             tb/unpu_top_tb.sv
```

`SCOPE = unpu_top_tb.dut` points the delays at the macro instance inside the testbench. `MAXIMUM` takes the slow corner (setup); repeat with `MINIMUM` for the fast corner (hold). Check the SDF log first: every instance should be annotated.

10. Troubleshooting

Symptom	Cause	Fix
`$fatal: could not open model/vectors/..._meta.txt -- run model/golden first`	No vectors, or not in the repo root	Run `./model/golden`; `cd` to the repo root
`xrun: command not found`	Cadence not loaded	`source ~/cad_cshrc`
`*E,UNDIDN`	Old testbench version	`git pull`
Two `*W` lines every run	Reported as IEEE 1800-2009 semantics notices	Informational
No verdict line at the end of the log	Stopped early, or a bounded wait hit its cap	Find the first `*E` or `FAIL` line
GLS: `DONE never observed`, `mem C = x`	`-nospecify` was used	Remove it
GLS: duplicate module / unresolved cell	Wrong cell model files	Only `tsl18fs120_scl.v`
Waiting for a licence	All Xcelium licences busy	`XRUN_FLAGS="-licqueue" bash scripts/run_xrun.sh`
Disk full	Old lib dirs (GLS alone is about 26 MB)	See the clean-up commands in chapter 11

11. Command reference card

Setup (once per session / clone)

```
        source ~/cad_cshrc
        gcc -std=c99 -Wall -Wextra -o model/golden model/golden.c && ./model/golden
```

RTL: all twelve testbenches

```
        xrun -sv -64bit -c -top unpu_top -f scripts/filelist.f -xmlibdirname sim/compile -l
logs/compile.log
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_pe_tb    -xmlibdirname sim/pe
-l logs/sim_pe.log    -f scripts/filelist.f tb/unpu_pe_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_grid_tb  -xmlibdirname sim/grid
-l logs/sim_grid.log  -f scripts/filelist.f tb/unpu_grid_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_skew_tb  -xmlibdirname sim/skew
-l logs/sim_skew.log  -f scripts/filelist.f tb/unpu_skew_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_stall_tb -xmlibdirname sim/stall
-l logs/sim_stall.log -f scripts/filelist.f tb/unpu_stall_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_buf_tb   -xmlibdirname sim/buf
-l logs/sim_buf.log   -f scripts/filelist.f tb/unpu_buf_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_dma_tb   -xmlibdirname sim/dma
-l logs/sim_dma.log   -f scripts/filelist.f tb/unpu_dma_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_seq_tb   -xmlibdirname sim/seq
-l logs/sim_seq.log   -f scripts/filelist.f tb/unpu_seq_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_csr_tb   -xmlibdirname sim/csr
-l logs/sim_csr.log   -f scripts/filelist.f tb/unpu_csr_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_apb_tb   -xmlibdirname sim/apb
-l logs/sim_apb.log   -f scripts/filelist.f tb/unpu_apb_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_top_tb   -xmlibdirname sim/top
-l logs/sim_top.log   -f scripts/filelist.f tb/unpu_top_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_ext1_tb  -xmlibdirname sim/ext1
-l logs/sim_ext1.log  -f scripts/filelist.f tb/unpu_ext1_tb.sv
        xrun -sv -64bit -access +r -timescale 1ns/1ps -top unpu_ext2_tb  -xmlibdirname sim/ext2
-l logs/sim_ext2.log  -f scripts/filelist.f tb/unpu_ext2_tb.sv
```

Regression, GUI, waveforms

```
        bash scripts/run_xrun.sh                    # ten frozen TBs
        bash scripts/run_xrun.sh ext1 ext2          # peer TBs
        <any command above> -gui                    # SimVision
        <any command above> -input sim/waves.tcl    # batch dump to waves.shm
        simvision waves.shm &
```

Gate-level (pre-layout)

```
        xrun -sv -64bit -access +r -timescale 1ns/1ps -notimingchecks -top unpu_top_tb -
xmlibdirname gls/top -l logs/gls_top.log syn/out/unpu_top_cn.dc.v
/storage/PDK_iitg/SCLPDK_V3.0_KIT/scl180/stdcell/fs120/4M1IL/verilog/vcs_sim_model/tsl18fs120_scl
.v tb/unpu_top_tb.sv
```

Verilator (laptop)

```
        verilator --binary --timing -Wno-fatal --top-module unpu_<tb>_tb --Mdir obj_dir/<tb>
rtl/*.sv tb/unpu_<tb>_tb.sv && ./obj_dir/<tb>/Vunpu_<tb>_tb
```

Check a log

```
        grep -E "ALL .*PASSED|ALL TESTS PASSED" logs/<log>;  grep -cE "\*E,|\*F," logs/<log>;
grep -c FAIL logs/<log>
```

Clean up (keeps the history files)

```
        rm -rf sim/*/worklib sim/*/run.lnx8664.* gls/top/worklib gls/top/run.lnx8664.* xrun_out
waves.shm .simvision
```